

LIVE PROJECT REPORT

Project Sentinel: Honeypot Alerts System

*Submitted in partial fulfillment of the requirements for
the award of the degree of*

Bachelor of Technology Computer Science and Engineering

Supervisor :
Mrs. Manisha
(Asst. Professor)

Submitted by:
1. Chayan Das (11824210002)
2. Aditya Kaushik (11824210008)
3. Deepanshu Semwal (11824210010)



SRM
UNIVERSITY
DELHI-NCR, SONEPAT

**SRM UNIVERSITY, DELHI-NCR, SONEPAT, HARYANA Plot
No. 39, Rajiv Gandhi Education City, Sonapat, Haryana 131029
MAY 2026**

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project entitled "Project Sentinel: Honeypot Alerts System" in partial fulfilment of the requirement for the award of the Degree of Bachelor of Technology in Computer Science and Engineering [in specialization with Cyber Security] of SRM University, Delhi-NCR, Sonapat, Haryana, (India) is an authentic record of our own work carried out under the supervision of **Mrs Manisha**, as Live project in 4th Semester during the academic year 2025-26. The matter presented in this project has not been submitted for the award of any other degree of this or any other Institute/University.

STUDENT 1 — **Chayan Das**
Registration No.: 11824210002

STUDENT 2 — **Aditya Kaushik**
Registration No.: 11824210008

STUDENT 3 — **Deepanshu Semwal**
Registration No.: 11824210010

CERTIFICATE

This is to certify that the "Project Sentinel: Honeypot Alerts System" project is a Bonafide work completed as a Live Project (Course Code: 21CS4114) in partial fulfilment for the award of the degree of Bachelor of Technology in CSE-Cyber Security under the guidance of **Mrs Manisha**. This project is submitted by **Chayan, Aditya, and Deepanshu** during the academic Semester IV of year 2025-2026 of SRM University, Delhi-NCR, Sonipat, Haryana.

Mrs. Manisha
(Asst. Professor)

ACKNOWLEDGEMENT

Most importantly, we would like to express our immense gratitude to our supervisor for patient guidance, encouragement, training, and advice provided throughout this project. The depth of knowledge we have acquired during the course of this work — spanning network security, API engineering, database design, and cloud deployment — will remain with us throughout our professional careers. We have been extremely fortunate to have a supervisor who cared deeply about the quality of our work and responded to our questions and queries promptly and thoughtfully. We also sincerely thank the Project Coordinator, Mrs Manisha, for her valuable guidance, continuous support, and encouragement throughout the project. His structured feedback during review presentations helped us significantly improve both the technical quality and the documentation of our work.

We would like to extend our indebtedness to the Project Committee Members of the Department of Computer Science and Engineering, SRM University Delhi-NCR, Sonapat, as we have benefited enormously from the constructive criticism and practical suggestions provided during project reviews. Their professional insights prompted us to think more critically about system security, scalability, and real-world deployment considerations.

We feel compelled to articulate our sincere thankfulness to Prof. M. Mohan, HOD, Department of Computer Science and Engineering, SRM University, for his encouragement and the department's commitment to providing students with live project opportunities that bridge theoretical knowledge with practical implementation.

Lastly, we are deeply indebted to all the teaching and non-teaching staff members of the university for helping us directly or indirectly throughout the course of our study and project work. We also thank our families for their patient support and encouragement during the demanding phases of this project.

— *Chayan, Aditya, and Deepanshu*

ABSTRACT

Project Sentinel is a real-time, multi-protocol honeypot-based intrusion detection and alerting system engineered to capture, analyse, and report malicious network activity. The increasing prevalence of automated cyber attacks — particularly brute-force SSH attacks and HTTP credential stuffing campaigns — motivates the need for proactive threat intelligence systems that can detect these attacks and alert administrators before damage occurs. Traditional security tools such as firewalls and intrusion prevention systems operate on known signatures and rules; honeypots complement these tools by providing insight into novel attack techniques and attacker behaviour.

The system architecture consists of three core components deployed via Docker Compose. First, an SSH Honeypot sensor implemented in Python 3 using the Paramiko library simulates a vulnerable SSH server on port 2222. It intercepts authentication attempts, captures attacker credentials (username and password), determines the attacker's geographic origin using the ipapi.com geolocation service, applies a sliding-window rate-limiting algorithm, and forwards structured JSON event data to the Central API. Second, an HTTP Honeypot sensor implemented using the Python Flask framework renders a convincingly styled administrative login portal on port 8080. It captures form-submitted credentials from automated scanners and manual attackers, applying identical rate-limiting logic, and forwards events to the Central API. Third, the Core API implemented in Go (Golang) using the Gin web framework serves as the event aggregation and alerting hub. It exposes REST endpoints for event ingestion (/api/v1/ingest), health monitoring (/health), and log retrieval (/logs), persists all events to a PostgreSQL (TimescaleDB-compatible) database, and dispatches multi-channel alerts via Gmail SMTP (using the gmail library) and the Telegram Bot API.

The system incorporates a comprehensive monitoring stack: Grafana dashboards connected to the PostgreSQL backend provide real-time visualizations of attack patterns, geographic distributions, and sensor health. Pre-provisioned Grafana alerting rules automatically trigger on brute-force patterns, multi-service attack campaigns, and abnormal traffic spikes.

Testing demonstrated successful end-to-end operation: SSH and HTTP attack simulations were correctly captured, persisted, and triggered alerts within seconds. The IP blocking mechanism successfully prevented resource exhaustion during high-volume simulated attacks. The system

was deployed on Render.com cloud infrastructure, demonstrating feasibility for low-cost, production-grade cloud deployment.

This project demonstrates practical application of network security principles, distributed systems design, RESTful API development, containerized infrastructure, and real-time monitoring — providing the development team with extensive hands-on experience in building production-grade cybersecurity infrastructure.

Keywords: Honeypot, Intrusion Detection System, SSH Brute-Force, HTTP Credential Stuffing, Golang, Python, Flask, Paramiko, PostgreSQL, TimescaleDB, Docker Compose, Grafana, Telegram Bot API, Email Alerting, Real-Time Security Monitoring, Cybersecurity.

TABLE OF CONTENTS

Title Page	
Candidate's Declaration	I
Certificate	II
Acknowledgement	III
Abstract	IV
List of Figures	X
List of Tables	XI
Chapter 1: Introduction	1
1.1 Background of the Study	1
1.2 Introduction to Honeypot Technology	2
1.2.1 Low-Interaction Honeypots	2
1.2.2 High-Interaction Honeypots	2
1.3 Problem Statement	3
1.4 Objectives of the Project	3
1.5 Scope of the Project	4
1.6 Significance of the Study	4
1.7 Applications of the Project	5
Chapter 2: Literature Review	6
2.1 Introduction	6
2.2 Honeypot Systems	6
2.2.1 Low-Interaction Honeypots	7
2.2.2 High-Interaction Honeypots	7

2.3 Existing Tools	8
2.3.1 Cowrie	8
2.3.2 Dionaea	9
2.3.3 Kippo	9
2.4 Limitations of Existing Systems	10
2.5 Research Gap	10
2.6 Proposed Solution	11
 Chapter 3: Methodology	 12
3.1 Introduction	12
3.2 System Approach	12
3.3 Architecture Overview	13
3.4 Modules of the System	14
3.5 Tools and Technologies Used	16
3.6 Workflow of the System	17
3.7 Advantages of Methodology	18
3.8 Limitations of Methodology	18
 Chapter 4: System Design	 19
4.1 Introduction	19
4.2 System Architecture	19
4.3 Data Flow Diagram (DFD)	20
4.4 Use Case Diagram	21
4.5 Sequence Diagram	22
4.6 Database Design	23
4.7 Interface Design	24

4.8 Security Considerations	25
4.9 Design Advantages	25
 Chapter 5: Implementation	 26
5.1 Introduction	26
5.2 Project Structure	27
5.3 Honeypot Implementation	29
5.4 Logging Implementation	31
5.5 Alert System Implementation	32
5.6 IP Blocking Implementation	33
5.7 Grafana Dashboard Integration	34
5.8 Key Features Implemented	35
5.9 Challenges Faced	36
5.10 Solutions to Challenges	36
 Chapter 6: Testing and Results	 37
6.1 Introduction	37
6.2 Testing Methodology	37
6.3 Test Cases	38
6.4 Testing of SSH Honeypot	39
6.5 Testing of HTTP Honeypot	40
6.6 Alert System Testing	41
6.7 IP Blocking Testing	42
6.8 Graphical Results	43
6.9 Performance Evaluation	44
6.10 Limitations Identified	44

6.11 Summary of Results	45
Chapter 7: Conclusion and Future Work	46
7.1 Conclusion	46
7.2 Achievements	47
7.3 Limitations	48
7.4 Future Scope	49
References	50
Appendix A: System Log Data	51
Appendix B: Database Structure	52
Appendix C: API Endpoints	53
Appendix D: Project Folder Structure	54
Appendix E: Commands Used	55
Appendix F: Alert System Workflow	56

LIST OF FIGURES

Figure 3.1 — System Architecture	13
Figure 3.2 — Workflow Diagram	17
Figure 4.1 — Data Flow Diagram	19
Figure 4.2 — Use Case Diagram	20
Figure 4.3 — Sequence Diagram	21
Figure 4.4 — Database Representation	22
Figure 5.1 — Project Structure	25
Figure 5.2 — SSH Honeypot	29
Figure 5.3 — HTTP Honeypot	31
Figure 5.4 — Logging Output	33
Figure 5.5 — Alert System	34
Figure 5.6 — IP Blocking	35
Figure 5.7 — Grafana Dashboard	36
Figure 6.1 — SSH Testing	40
Figure 6.2 — HTTP Testing	40
Figure 6.3 — Alert Testing	41
Figure 6.4 — IP Blocking Testing	42
Figure 6.5 — Attack Graph	43
Figure 6.6 — Login Frequency	43
Figure 6.7 — HTTP Trends	43
Figure 8.1 — HTTP Attack Logs	49
Figure 8.2 — SSH Attack Logs	50
Figure 8.3 — Project Folder Structure	53
Figure 8.4 — Alert System Workflow	56

LIST OF TABLES

Table 3.1 — Tools and Technologies 16

Table 6.1 — Test Cases 38

Table B.1 — Attack Logs Table 52

Table B.2 — Blocked IPs Table 52

Table C.1 — API Endpoints 53

Table E.1 — Commands Used 55

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

In the modern digital era, the rapid growth of internet usage has significantly increased the number of cyber threats and attacks. Organizations, institutions, and individuals rely heavily on networked systems for communication, data storage, and service delivery. However, this dependency also exposes systems to various types of cyberattacks such as brute-force attacks, unauthorized access, malware injections, and data breaches.

Traditional security mechanisms such as firewalls, antivirus software, and intrusion prevention systems are designed to protect systems from known threats. These systems mainly operate on predefined rules and signatures, which makes them less effective against new and unknown attack patterns. As attackers continuously evolve their techniques, there is a need for more advanced and proactive security solutions.

One such approach is the use of honeypots. A honeypot is a decoy system that is intentionally designed to attract attackers by simulating vulnerable services. Unlike traditional defense mechanisms, honeypots do not directly prevent attacks but instead monitor and analyze attacker behavior. This allows security professionals to gain valuable insights into attack patterns and improve overall system security.

The "Project Sentinel - Honeypot Monitoring System" is developed based on this concept. It aims to create a controlled environment where attackers can interact with a simulated system, while their actions are logged and analyzed in real time.

1.2 Introduction to Honeypot Technology

A honeypot is a cybersecurity mechanism that acts as a trap for malicious users. It is designed to appear as a legitimate system with vulnerabilities, encouraging attackers to engage with it. The primary purpose of a honeypot is not to protect the system directly but to collect information about attack techniques and behavior.

Honeypots can be classified into two main categories:

1.2.1 Low-Interaction Honeypots

These honeypots simulate limited services and interactions. They are easy to deploy and maintain but provide less detailed information about attacker activities.

1.2.2 High-Interaction Honeypots

These honeypots provide a real operating environment where attackers can fully interact with the system. They offer more detailed insights but are more complex and risky to manage.

The system developed in this project is primarily a **low-interaction honeypot**, focusing on simulating SSH services and logging attack attempts.

1.3 Problem Statement

Despite the availability of various cybersecurity tools, many organizations still face challenges in detecting and analyzing cyber threats effectively. Traditional systems often fail to provide detailed insights into attacker behavior and may not detect new types of attacks.

The key problems identified are:

- Lack of real-time monitoring of attacker activities
- Inability to analyze unknown or emerging threats
- Limited visibility into attack patterns
- Absence of automated response mechanisms

To address these issues, there is a need for a system that can:

- Attract attackers intentionally
- Monitor their behavior in real time
- Generate alerts for suspicious activities
- Automatically respond to threats

1.4 Objectives of the Project

The primary objectives of the "Project Sentinel - Honeypot Monitoring System" are:

- To design and develop a honeypot system that simulates vulnerable network services
- To capture and log attacker activities such as login attempts and commands
- To implement a real-time alert system, including email notifications for detected attacks
- To develop an automated IP blocking mechanism to prevent repeated attacks
- To integrate a visualization dashboard for monitoring attack patterns
- To analyze collected data for improving system security

1.5 Scope of the Project

The scope of this project includes the design, development, and implementation of a honeypot system that focuses on monitoring and analyzing cyberattacks.

The system is limited to:

- Simulation of SSH-based attacks
- Logging and monitoring of attacker activities
- Real-time alert generation
- Temporary blocking of malicious IP addresses

However, the system can be extended in the future to include:

- Multiple honeypot services (FTP, HTTP, etc.)
- Advanced machine learning-based attack detection
- Cloud-based deployment for scalability

1.6 Significance of the Study

This project is significant as it provides a proactive approach to cybersecurity. Instead of only defending against attacks, the system actively attracts attackers and studies their behavior.

The benefits include:

- Improved understanding of attack techniques
- Enhanced threat detection capabilities
- Real-time monitoring of suspicious activities
- Practical implementation of cybersecurity concepts

Additionally, the project serves as a valuable learning experience for understanding real-world cyber threats and defense mechanisms.

1.7 Applications of the Project

The honeypot monitoring system can be applied in various domains, including:

- Network security monitoring
- Cybersecurity research and analysis
- Educational purposes for learning ethical hacking
- Threat intelligence collection
- Enterprise security systems

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

A literature review is an essential part of any project as it provides an overview of existing research, technologies, and methodologies related to the topic. It helps in understanding the current state of development in the field and identifying the gaps that need to be addressed.

In the domain of cybersecurity, several approaches have been developed to detect and prevent unauthorized access. These include firewalls, intrusion detection systems (IDS), intrusion prevention systems (IPS), and honeypots. Each of these systems has its own advantages and limitations.

Honeypot technology has gained significant attention in recent years due to its ability to act as a proactive security mechanism. Instead of simply blocking attacks, honeypots allow attackers to interact with a decoy system, thereby providing valuable insights into their behavior, tools, and techniques.

This chapter reviews existing honeypot systems, related tools, and research studies to understand their working principles, advantages, and limitations. It also highlights the need for an improved system such as the "Project Sentinel - Honeypot Monitoring System".

2.2 Honeypot Systems

Honeypots are specialized security systems designed to attract attackers by simulating vulnerable environments. They are widely used for research, monitoring, and threat analysis purposes.

Honeypots can be broadly classified into the following types:

2.2.1 Low-Interaction Honeypots

Low-interaction honeypots simulate only limited services and functionalities. They do not provide a complete operating environment but instead mimic certain aspects of a system.

Characteristics:

- Easy to deploy and maintain
- Low resource consumption
- Limited interaction with attackers
- Reduced risk of system compromise

Advantages:

- Safe to use as attackers cannot fully control the system
- Suitable for beginners and educational purposes
- Requires minimal configuration

Disadvantages:

- Provides limited information about attacker behavior
- Cannot capture complex attack patterns

In this project, a low-interaction honeypot approach is used to simulate an SSH service and capture login attempts.

2.2.2 High-Interaction Honeypots

High-interaction honeypots provide a real operating system environment where attackers can interact freely. These systems are designed to capture detailed information about attack techniques.

Characteristics:

- Realistic system environment
- High level of interaction
- Complex setup and maintenance

Advantages:

- Provides detailed insights into attacker behavior

- Captures advanced attack techniques
- Useful for research and forensic analysis

Disadvantages:

- High risk if not properly secured
- Requires more resources
- Difficult to manage

Although high-interaction honeypots provide more detailed data, they are not used in this project due to complexity and risk factors.

2.3 Existing Tools

Several tools and frameworks have been developed for implementing honeypot systems. These tools provide different levels of interaction and functionality.

2.3.1 Cowrie

Cowrie is a widely used open-source SSH and Telnet honeypot. It is designed to log brute-force attacks and capture attacker commands.

Features:

- Emulates SSH and Telnet services
- Logs login attempts and commands
- Supports file download monitoring
- Provides detailed attack logs

Limitations:

- Requires configuration and setup
- Limited built-in visualization features

2.3.2 Dionaea

Dionaea is a malware honeypot designed to capture and analyze malware samples. It focuses on collecting malicious files and studying their behavior.

Features:

- Captures malware samples
- Supports multiple protocols
- Provides analysis tools

Limitations:

- Complex setup
- Limited focus on login-based attacks

2.3.3 Kippo

Kippo is an older SSH honeypot that records attacker sessions. It has been widely used for research purposes.

Features:

- Logs attacker commands
- Records session activity
- Provides basic logging

Limitations:

- No longer actively maintained
- Limited features compared to modern tools

2.4 Limitations of Existing Systems

Although existing honeypot tools provide useful features, they also have several limitations:

- Lack of real-time alert systems
- Limited integration with visualization dashboards
- Absence of automated response mechanisms
- Complex setup and configuration
- Limited scalability

Most existing systems focus primarily on logging attacker activity but do not provide immediate response or visualization capabilities.

2.5 Research Gap

Based on the analysis of existing systems, the following research gaps have been identified:

- Need for real-time monitoring and alert systems
- Lack of automated IP blocking mechanisms
- Limited user-friendly dashboards for visualization
- Absence of integrated systems combining detection, logging, and response

There is a clear need for a system that not only detects attacks but also responds to them automatically and provides real-time insights.

2.6 Proposed Solution

The "Project Sentinel - Honeypot Monitoring System" is designed to address the limitations identified in existing systems.

The proposed system offers the following features:

- Simulation of SSH-based honeypot environment
- Real-time logging of attacker activities
- Alert system with notification capabilities (including email alerts)

- Automated IP blocking for repeated attacks
- Integration with visualization dashboard (Grafana)

Unlike traditional honeypot systems, this project combines detection, monitoring, visualization, and response into a single integrated solution.

CHAPTER 3: METHODOLOGY

3.1 Introduction

The methodology defines the systematic approach followed to design, develop, and implement the "Project Sentinel - Honeypot Monitoring System". It provides a clear understanding of how the system operates, the sequence of operations involved, and the interaction between different components.

The project follows a modular and structured approach to ensure that each component performs a specific function efficiently. The methodology focuses on simulating a vulnerable environment, capturing attacker behavior, generating alerts, and responding to threats automatically.

Unlike traditional security systems that primarily focus on prevention, the methodology used in this project emphasizes **monitoring, analysis, and response**. This approach helps in understanding real-world attack patterns and improving overall system security.

3.2 System Approach

The system is based on a **decoy-based security approach**, where a fake service is deployed to attract attackers. The goal is not to directly block attackers initially, but to observe their behavior and collect valuable information.

The overall approach of the system can be divided into multiple stages:

3.2.1 Deployment of Honeypot Service

A simulated SSH service is deployed on the system. This service appears as a legitimate entry point for attackers, encouraging them to attempt login operations.

3.2.2 Monitoring Incoming Connections

The system continuously listens for incoming connections. Whenever an attacker attempts to connect, the system records the connection details such as IP address and timestamp.

3.2.3 Capturing Login Attempts

The honeypot captures the username and password entered by the attacker. These credentials are stored for further analysis.

3.2.4 Logging Attacker Activity

All attacker activities, including login attempts and commands, are recorded in log files. This helps in understanding attack patterns.

3.2.5 Alert Generation

When suspicious behavior is detected, such as repeated login attempts, the system generates alerts. These alerts notify the system administrator about potential threats.

3.2.6 Automated Response

If the number of failed attempts exceeds a predefined threshold, the system automatically blocks the attacker's IP address using firewall rules.

3.2.7 Data Visualization

The collected data is visualized using a dashboard, allowing the user to monitor attack trends and system activity in real time.

3.3 Architecture Overview

The system architecture consists of multiple interconnected modules that work together to detect and respond to cyber threats.

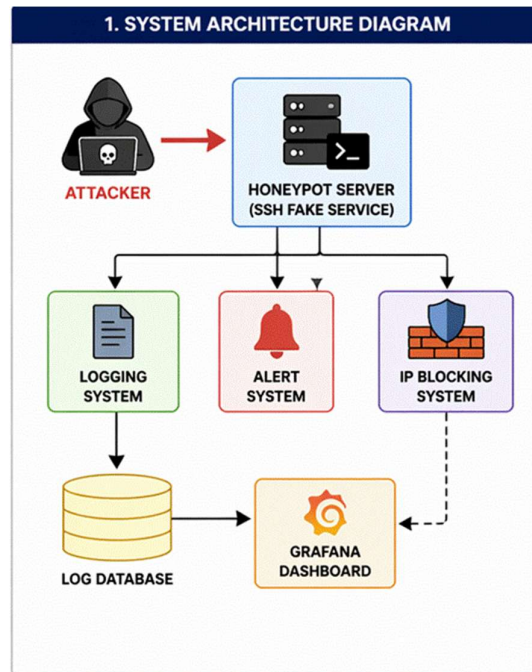


Fig. 3.1

Explanation of Architecture

- The attacker interacts with the honeypot server.
- The honeypot captures all activities and sends data to the logging module.
- The logging module stores the data in files or databases.
- The alert system analyzes the data and generates notifications.
- The firewall module blocks malicious IPs.
- The dashboard displays all collected information visually.

3.4 Modules of the System

The system is divided into multiple modules to ensure modularity and efficient functioning.

3.4.1 Honeypot Module

The honeypot module is the core component of the system. It simulates a vulnerable SSH service that attracts attackers.

Functions:

- Listens for incoming connections
- Simulates login interface
- Accepts user inputs
- Captures credentials

Working:

When an attacker attempts to connect, the honeypot presents a fake login prompt. The entered username and password are captured and stored without granting actual access.

3.4.2 Logging Module

The logging module is responsible for storing all attacker activities in a structured format.

Data Logged Includes:

- IP address
- Timestamp
- Username
- Password
- Commands executed

Importance:

The logs provide valuable information for analyzing attack patterns and identifying malicious behavior.

3.4.3 Alert Module

The alert module plays a critical role in notifying the user about suspicious activities.

Features:

- Real-time alert generation
- Detection of repeated login attempts
- Identification of malicious behavior

Notification Methods:

- Email notifications sent to the system administrator
- Console-based alerts for immediate visibility

Working:

When the number of failed login attempts exceeds a threshold, the system triggers an alert and informs the user about the attack.

3.4.4 IP Blocking Module

The IP blocking module provides an automated response mechanism to prevent repeated attacks.

Working:

- Tracks the number of attempts from each IP
- Compares with predefined threshold
- Blocks IP if threshold exceeded

Method Used:

- Uses firewall rules (iptables) to block IP addresses

Benefits:

- Prevents continuous attacks
- Enhances system security
- Reduces load on the system

3.4.5 Visualization Module

This module provides graphical representation of collected data using a dashboard.

Features:

- Displays number of attacks
- Shows IP distribution
- Tracks attack frequency
- Time-based analysis

Tool Used:

- Grafana

Importance:

Visualization helps in understanding attack trends and making informed decisions.

3.5 Tools and Technologies Used

The system is implemented using a combination of programming languages and tools.

Table 3.1 :

Tool/Technology	Purpose
Python	Core development
Flask	Web interface
Grafana	Visualization
Logging Libraries	Store attack data
Linux Firewall	IP blocking
SSH Protocol	Honeypot simulation

3.6 Workflow of the System

The workflow describes how the system operates step-by-step.

1. Attacker attempts to connect to honeypot
2. Connection is accepted by the system
3. Login credentials are captured
4. Data is stored in logs
5. System checks for suspicious behavior
6. Alert is generated if needed
7. IP is blocked if threshold exceeded
8. Data is displayed on dashboard

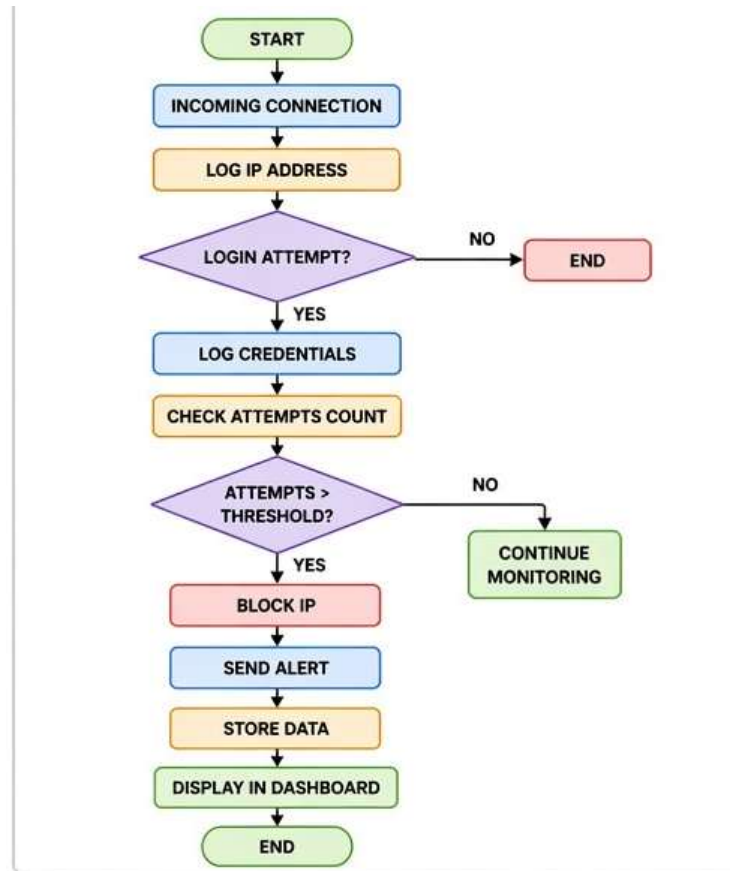


Fig. 3.2

3.7 Advantages of Methodology

- Provides proactive security approach
- Enables real-time monitoring
- Automates threat response
- Improves understanding of attack patterns
- Easy to extend and modify

3.8 Limitations of Methodology

- Limited to simulated services
- Cannot detect attacks outside honeypot
- Requires proper configuration
- Dependent on system resources

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

System design is a crucial phase in the development of any project as it defines the structure, components, and interactions within the system. It provides a blueprint for implementation and ensures that all modules work together efficiently.

In the "Project Sentinel - Honeypot Monitoring System", the system design focuses on creating a modular and scalable architecture. Each component is designed to perform a specific function such as capturing attacker activity, logging data, generating alerts, blocking malicious IPs, and visualizing information.

The design ensures:

- Clear separation of modules
- Efficient data flow
- Easy scalability and maintenance
- Real-time monitoring and response

4.2 System Architecture

The system begins with an attacker attempting to connect to the honeypot server. The honeypot server simulates a vulnerable environment and captures all incoming interactions.

The captured data is passed to the logging module, which records all relevant information such as IP address, timestamp, and login credentials.

The alert system continuously monitors the logs and generates alerts when suspicious activity is detected. At the same time, the IP blocking module takes action against repeated attacks by blocking malicious IP addresses.

The stored data is visualized through the Grafana dashboard, providing real-time insights into system activity.

4.3 Data Flow Diagram (DFD)

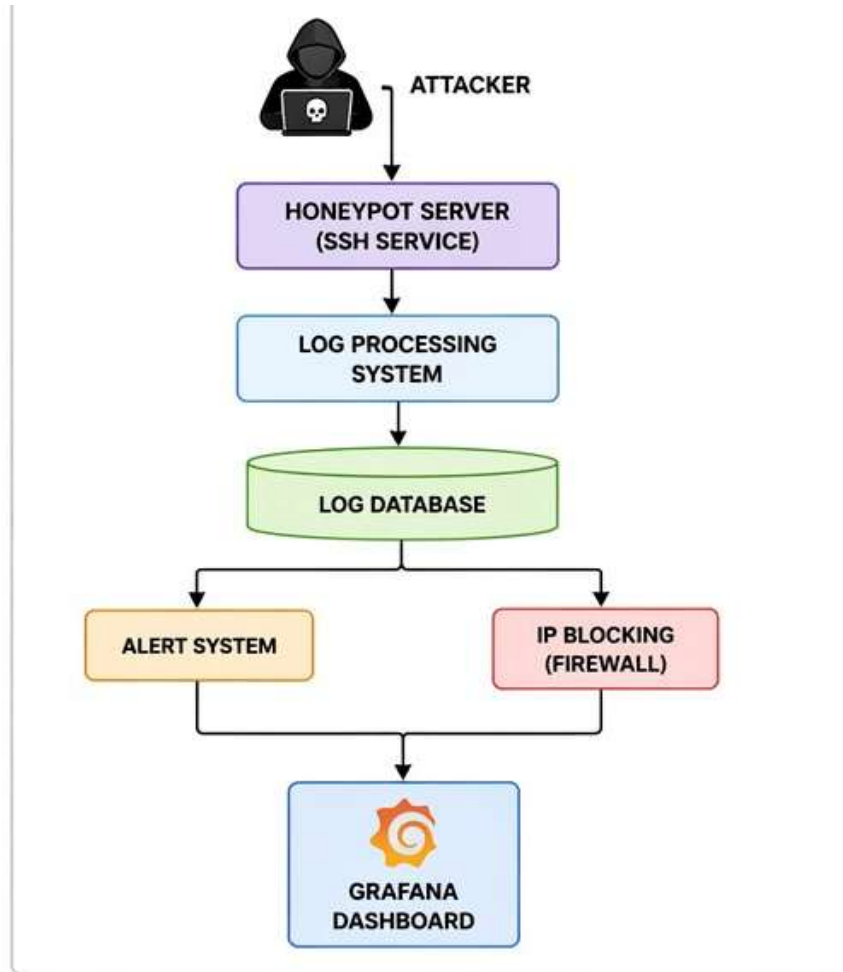


Fig. 4.1

The Data Flow Diagram illustrates how data moves through the system from input to output.

Explanation of DFD

- The attacker initiates a connection request.
- The honeypot server processes the request and captures input data.
- The logging module records all activities.
- The alert system analyzes the data for suspicious patterns.
- If malicious behavior is detected, the IP blocking module is triggered.
- The collected data is visualized for monitoring and analysis.

4.4 Use Case Diagram

The Use Case Diagram represents interactions between users (actors) and the system.

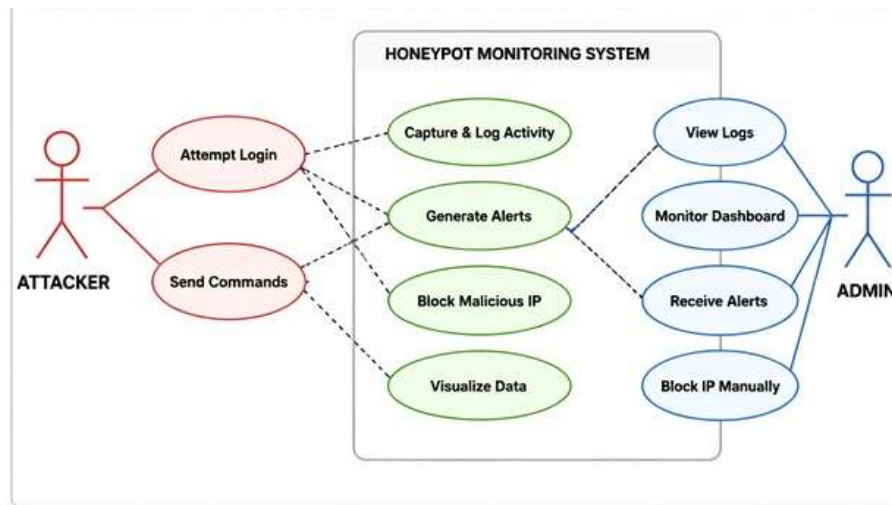


Fig. 4.2

Actors:

- Attacker
- System Administrator

Use Cases:

- Attempt Login
- Monitor Logs
- Receive Alerts
- Block IP
- View Dashboard

Explanation of Use Case Diagram

The attacker interacts with the honeypot by attempting to log in. These actions are captured by the system.

The system administrator monitors logs, receives alerts, and observes attack patterns through the dashboard. The administrator may also take manual actions if required, although most responses are automated.

4.5 Sequence Diagram

The sequence diagram shows the order of interactions between different components in the system.

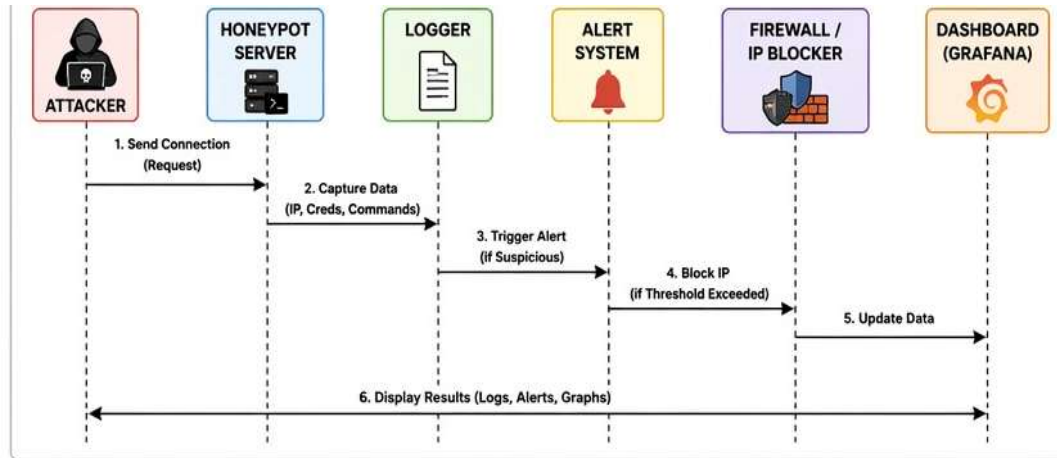


Fig. 4.3

Explanation of Sequence Diagram

1. The attacker initiates a connection.
2. The honeypot server accepts the request and captures input.
3. The logging module stores the data.
4. The alert system checks for suspicious behavior.
5. If necessary, the IP blocking module blocks the attacker.
6. The dashboard displays the collected data.

4.6 Database Design

Although the system primarily uses log files, the data is structured in a way similar to a database.

Data Fields:

- IP Address
- Timestamp
- Username
- Password

- Command Executed
- Status (Success/Failed)

Sample Data Representation

/ event_id	timestamp	/ sensor_id	/ sensor_location	/ source_ip	/ vector	/ interaction_level	/ username_attempt	/ password_attempt
24285a2a-abbf-4e84	2026-04-19 13:39:25	ssh-eu-1	london	127.0.0.1	ssh	low	test	qwerty234
d6c43b52-ab72-4dd	2026-04-19 13:39:32	ssh-eu-1	london	127.0.0.1	ssh	low	test	qwerty345
13695c2d-63e2-453f	2026-04-19 13:43:17	ssh-eu-1	london	127.0.0.1	ssh	low	test	qwerty123
6cd9992a-d7e0-409	2026-04-19 13:43:23	ssh-eu-1	london	127.0.0.1	ssh	low	test	asdf1234
b0485ed1-0809-48e	2026-04-19 13:43:29	ssh-eu-1	london	127.0.0.1	ssh	high	test	zxcv1234
c1a22d0a-a3dd-479f	2026-04-19 21:37:24	http-eu-1	london	127.0.0.1	http	low	user	admin
ebb2af5d-c4f4-4010	2026-04-19 21:38:02	http-eu-1	london	127.0.0.1	http	low	user	admin
acaa3067-7716-498c	2026-04-19 21:38:10	http-eu-1	london	127.0.0.1	http	low	user	admin
9943dd45-f124-473f	2026-04-19 21:38:11	http-eu-1	london	127.0.0.1	http	low	user	admin

Fig. 4.4

Explanation

The structured format of logs helps in:

- Easy data retrieval
- Efficient analysis
- Integration with visualization tools

4.7 Interface Design

The system provides two main interfaces:

1. Command-Line Interface (CLI)

- Displays logs and alerts
- Used for monitoring system activity

2. Dashboard Interface (Grafana)

- Provides graphical visualization
- Displays attack patterns and statistics

Dashboard Features:

- Attack count visualization
- IP tracking
- Time-based analysis
- Real-time monitoring

4.8 Security Considerations

Security is an important aspect of system design, especially in a honeypot system.

Measures Taken:

- Isolation of honeypot environment
- Controlled interaction with attackers
- Secure storage of logs
- Prevention of system misuse

Importance:

These measures ensure that:

- The system remains safe from exploitation
- Attackers cannot compromise the actual system
- Data integrity is maintained

4.9 Design Advantages

- Modular architecture for easy development
- Scalable system design
- Real-time monitoring and response
- Easy integration with external tools

CHAPTER 5: IMPLEMENTATION

5.1 Introduction

The implementation phase is where the theoretical design of the system is converted into a working model. In this project, the "Project Sentinel - Honeypot Monitoring System" is implemented using Python and various supporting tools.

The system is developed using a modular approach, where each component such as the honeypot server, logging system, alert mechanism, IP blocking module, and visualization system is implemented separately and then integrated.

The implementation focuses on:

- Real-time interaction handling
- Efficient logging and storage
- Automated alert generation
- Secure and controlled response mechanisms

The overall implementation ensures that the system behaves like a realistic environment to attract attackers while maintaining complete control over their activities.

5.2 Project Structure

The "Project Sentinel - Honeypot Monitoring System" is organized into multiple directories and modules to ensure a clean, modular, and scalable architecture. Each folder in the project represents a specific functional component of the system, allowing better maintainability and easier debugging.

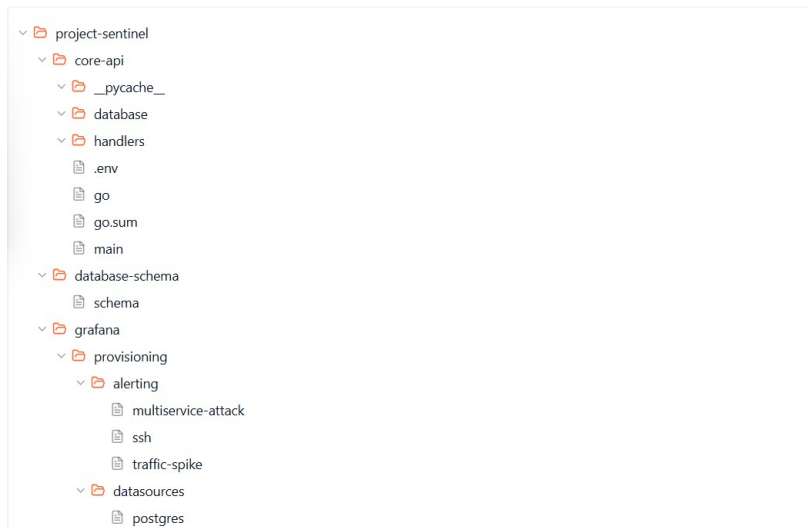


Fig. 5.1(a)

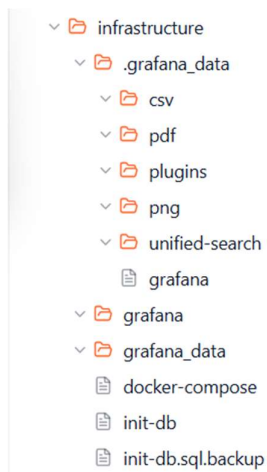


Fig. 5.1(b)

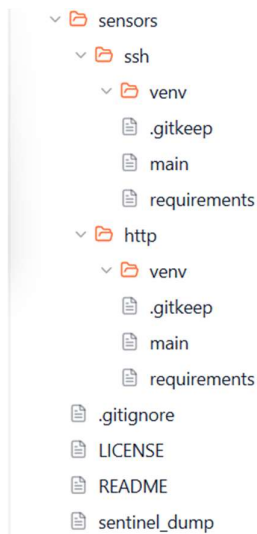


Fig. 5.1(c)

5.2.1 Core API Module

The **core-api** directory acts as the backend of the system. It is responsible for handling data processing, managing requests, and coordinating between different modules.

Subcomponents:

- **database/** → Handles database interactions and storage of attack data
- **handlers/** → Contains logic for processing incoming requests and events
- **main** → Entry point of the backend service

- **.env** → Stores environment variables such as database credentials
- **go / go.sum** → Dependency management files (indicates backend is written in Go)

Functionality:

This module collects data from sensors, processes it, and stores it in a structured format. It also provides APIs for integration with visualization tools like Grafana.

5.2.2 Database Schema Module

The **database-schema** directory defines the structure of the database used in the system.

Contents:

- **schema** → Contains SQL schema definitions

Functionality:

This module ensures that all collected data (such as attack logs, IP addresses, timestamps) is stored in a structured and consistent manner. It helps maintain data integrity and supports efficient querying.

5.2.3 Grafana Module

The **grafana** directory is responsible for configuring the monitoring dashboard.

Subcomponents:

- **provisioning/** → Contains configuration files for automatic setup
 - **alerting/** → Defines alert rules such as:
 - ❖ multi-service attack detection
 - ❖ SSH attack alerts
 - ❖ traffic spike alerts
 - **datasources/** → Defines data sources such as PostgreSQL

Functionality:

This module automatically sets up Grafana dashboards and alert rules. It enables real-time visualization of attack data and helps in monitoring system activity efficiently.

5.2.4 Infrastructure Module

The **infrastructure** directory manages system-level configurations and deployment.

Subcomponents:

- **grafana_data/** → Stores Grafana-related data
 - csv, pdf, png → Exported reports and visual data
 - plugins → Grafana plugins
- **unified-search/** → Provides search capabilities across stored data
- **docker-compose** → Used for containerized deployment
- **init-db / init-db.sql.backup** → Database initialization scripts

Functionality:

This module ensures that the entire system can be deployed and managed efficiently. It supports containerization using Docker and maintains persistent data storage.

5.2.5 Sensors Module

The **sensors** directory contains the honeypot implementations that interact directly with attackers.

Subcomponents:

1. SSH Sensor

- Simulates SSH service
- Captures login attempts
- Logs attacker credentials

2. HTTP Sensor

- Simulates web-based attacks
- Captures HTTP requests

Each sensor contains:

- **main** → Entry point
- **requirements** → Dependencies
- **venv** → Virtual environment

Functionality:

Sensors act as the entry point for attackers. They capture all interactions and forward the data to the core API for processing.

5.2.6 Root-Level Files

- **.gitignore** → Specifies files ignored by version control
- **LICENSE** → Defines project licensing
- **README** → Contains project documentation
- **sentinel_dump** → Backup or stored data dump

5.2.7 Overall Working of Project Structure

The system operates in a layered architecture:

1. Sensors (SSH/HTTP) capture attacker activity
2. Data is sent to the Core API
3. Core API processes and stores data in the database
4. Grafana retrieves data and displays it visually
5. Infrastructure manages deployment and storage

5.3 Honeypot Implementation (SSH and HTTP Server)

The honeypot system in this project is designed to simulate multiple services in order to attract different types of attackers. The implementation includes both an SSH server and an HTTP server, making the system capable of handling multiple attack vectors.

5.3.1 SSH Honeypot Implementation

The SSH honeypot simulates a secure shell service that allows attackers to attempt login operations.

Working Principle

The SSH server listens on a specific port and waits for incoming connections. When an attacker connects, the system presents a fake login interface and captures the entered credentials.

Sample Code :

```
import socket

server = socket.socket()

server.bind(("0.0.0.0", 2222))

server.listen(5)

while True:
```

```
client, addr = server.accept()

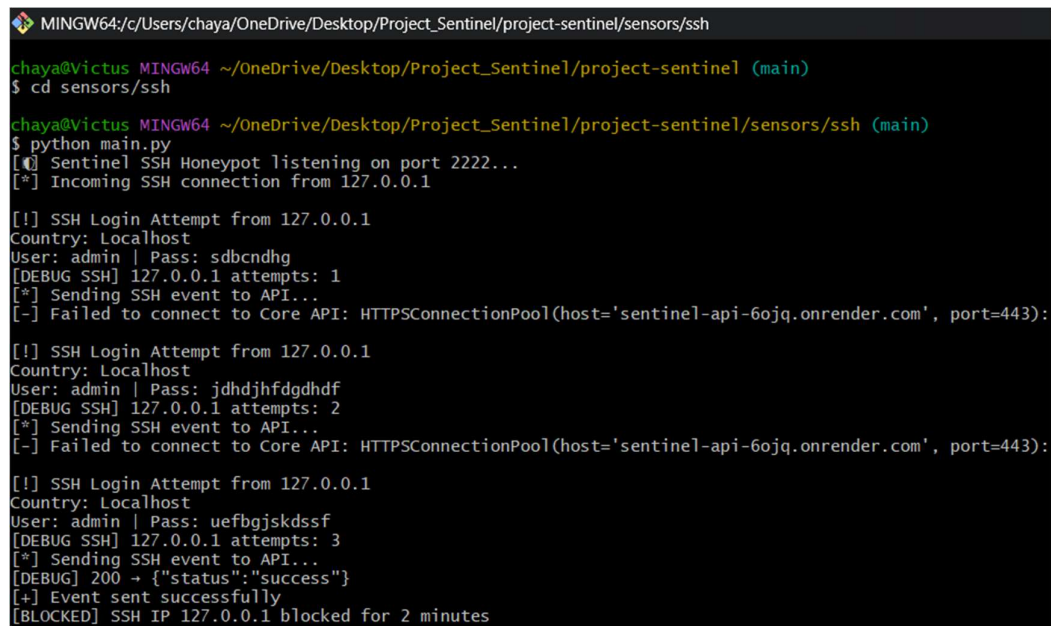
print("Connection from:", addr)
```

Explanation

- The server is created using the socket module
- It binds to port 22 (default SSH port)
- It continuously listens for incoming connections
- When a connection is established, the attacker's IP address is captured

Functionality

- Captures username and password attempts
- Logs attacker IP and activity
- Simulates a real SSH environment



```
MINGW64:/c:/Users/chaya/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh
chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ cd sensors/ssh
chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh (main)
$ python main.py
[!] Sentinel SSH Honeypot listening on port 2222...
[*] Incoming SSH connection from 127.0.0.1

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: sdbcmdhg
[DEBUG SSH] 127.0.0.1 attempts: 1
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: jdhdjhfdgdhdf
[DEBUG SSH] 127.0.0.1 attempts: 2
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: uefbgjskdssf
[DEBUG SSH] 127.0.0.1 attempts: 3
[*] Sending SSH event to API...
[DEBUG] 200 + {"status": "success"}
[+] Event sent successfully
[BLOCKED] SSH IP 127.0.0.1 blocked for 2 minutes
```

Fig. 5.2

```
MINGW64/c/Users/chaya/OneDrive/Desktop/Project_Sentinel/project-sentinel

chaya@victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ ssh-keygen -R "[localhost]:2222"
Host [localhost]:2222 not found in /c/Users/chaya/.ssh/known_hosts

chaya@victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ ssh admin@localhost -p 2222
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
RSA key fingerprint is: SHA256:rsyQPah5fd71Fy0Tf/7cfsScTyufzvzroSKqdk1GrQ
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
admin@localhost's password:
jdhjdjPermission denied, please try again.
admin@localhost's password:
Permission denied, please try again.
admin@localhost's password:
admin@localhost: Permission denied (password).
```

Figure 5.2(b)

5.3.2 HTTP Honeytrap Implementation

The HTTP honeytrap simulates a web server to capture web-based attacks such as malicious requests, scanning attempts, and injection attacks.

Working Principle

The HTTP server listens for incoming HTTP requests. When an attacker sends a request, the system captures details such as:

- Request method (GET, POST)
- URL accessed
- Headers
- Payload data

Sample Code :

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class HoneytrapHandler(BaseHTTPRequestHandler):

    def do_GET(self):

        print(f'Request from {self.client_address}')

        self.send_response(200)

        self.end_headers()

        self.wfile.write(b'Welcome to the server')
```

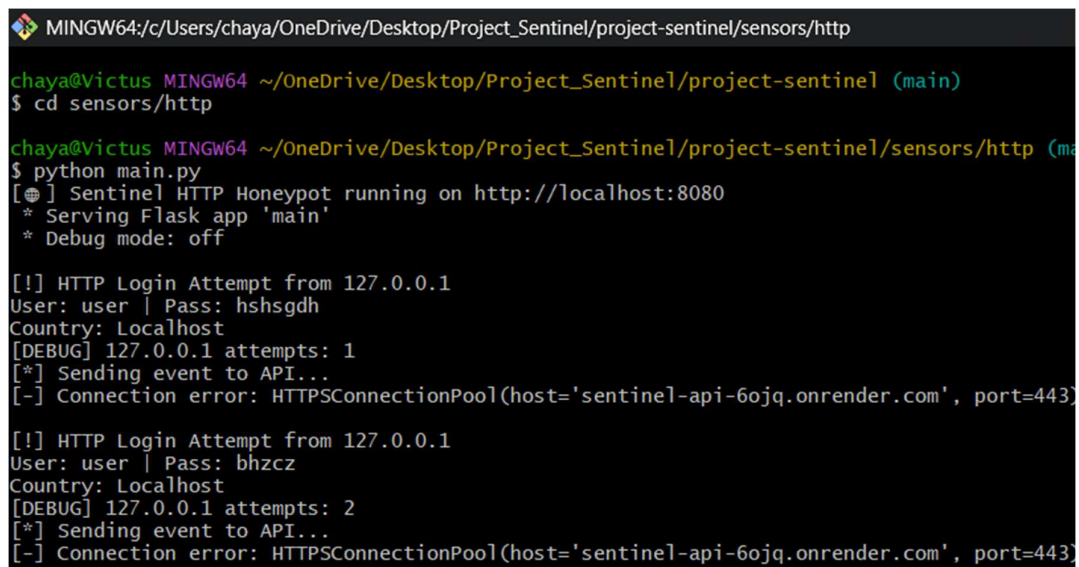
```
server = HTTPServer(("0.0.0.0", 80), HoneyPotHandler)
server.serve_forever()
```

Explanation

- A custom handler is created to process HTTP requests
- The server listens on port 80 (default HTTP port)
- Each request is captured along with the attacker's IP
- The system responds with a fake webpage

Functionality

- Captures HTTP requests and payloads
- Logs attacker activity
- Detects suspicious access patterns
- Helps identify web-based attacks



```

MINGW64:/c/Users/chaya/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/http
chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ cd sensors/http
chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/http (main)
$ python main.py
[+] Sentinel HTTP HoneyPot running on http://localhost:8080
* Serving Flask app 'main'
* Debug mode: off

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: hshsgdh
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 1
[*] Sending event to API...
[-] Connection error: HTTPConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443)

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: bhzc
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 2
[*] Sending event to API...
[-] Connection error: HTTPConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443)

```

Figure 5.3

5.3.3 Multi-Service HoneyPot Advantage

By combining both SSH and HTTP honeypots, the system becomes more effective in detecting a wide range of attacks.

Advantages:

- Captures both network-level and web-based attacks
- Provides better attack coverage
- Improves threat intelligence
- Makes the system more realistic to attackers

5.3.4 Integration with System

Both SSH and HTTP servers are integrated with the logging, alert, and IP blocking modules.

Workflow:

1. Attacker interacts with SSH or HTTP service
2. Data is captured and logged
3. System checks for suspicious behavior
4. Alerts are generated if necessary
5. IP is blocked if threshold exceeded

5.4 Logging Implementation

The logging module is responsible for recording all attacker activities.

Working Principle

Whenever an attacker attempts to log in, the system captures the entered username and password and stores them in a log file along with the IP address and timestamp.

Sample Code :

```
def log_attack(ip, username, password):
    with open("logs.txt", "a") as file:
        file.write(f"{ip} - {username} - {password}\n")
```

Explanation

- Opens a log file in append mode

- Writes attacker details into the file
- Ensures data persistence for analysis

<i>A</i> event_id	⌚ timestamp	<i>A</i> sensor_id	<i>A</i> sensor_location	<i>A</i> source_ip	<i>A</i> vector	<i>A</i> interaction_level	<i>A</i> username_attempt	<i>A</i> password
24285a2a-abbf-4e84	2026-04-19 13:39:25	ssh-eu-1	london	127.0.0.1	ssh	low	test	qwerty234
d6c43b52-ab72-4dd	2026-04-19 13:39:32	ssh-eu-1	london	127.0.0.1	ssh	low	test	qwerty345
13695c2d-63e2-453f	2026-04-19 13:43:17	ssh-eu-1	london	127.0.0.1	ssh	low	test	qwerty123
6cd9992a-d7e0-409	2026-04-19 13:43:23	ssh-eu-1	london	127.0.0.1	ssh	low	test	asdf1234
b0485ed1-0809-48e	2026-04-19 13:43:29	ssh-eu-1	london	127.0.0.1	ssh	high	test	zxcv1234
c1a22d0a-a3dd-479f	2026-04-19 21:37:24	http-eu-1	london	127.0.0.1	http	low	user	admin
ebb2af5d-c4f4-4010	2026-04-19 21:38:02	http-eu-1	london	127.0.0.1	http	low	user	admin
acaa3067-7716-498c	2026-04-19 21:38:10	http-eu-1	london	127.0.0.1	http	low	user	admin
9943dd45-f124-473f	2026-04-19 21:38:11	http-eu-1	london	127.0.0.1	http	low	user	admin

Figure 5.4

5.5 Alert System Implementation

The alert system is implemented to notify the system administrator about suspicious activities.

Working Principle :

The system monitors login attempts from each IP address. If the number of attempts exceeds a predefined threshold, an alert is triggered.

Features

- Real-time alert generation
- Detection of repeated login attempts
- Email notifications to the administrator
- Console-based alerts

Sample Logic :

```
if attempts > threshold:
    send_alert(ip)
```

Explanation

- Tracks number of attempts

- Compares with threshold value
- Triggers alert when limit is exceeded

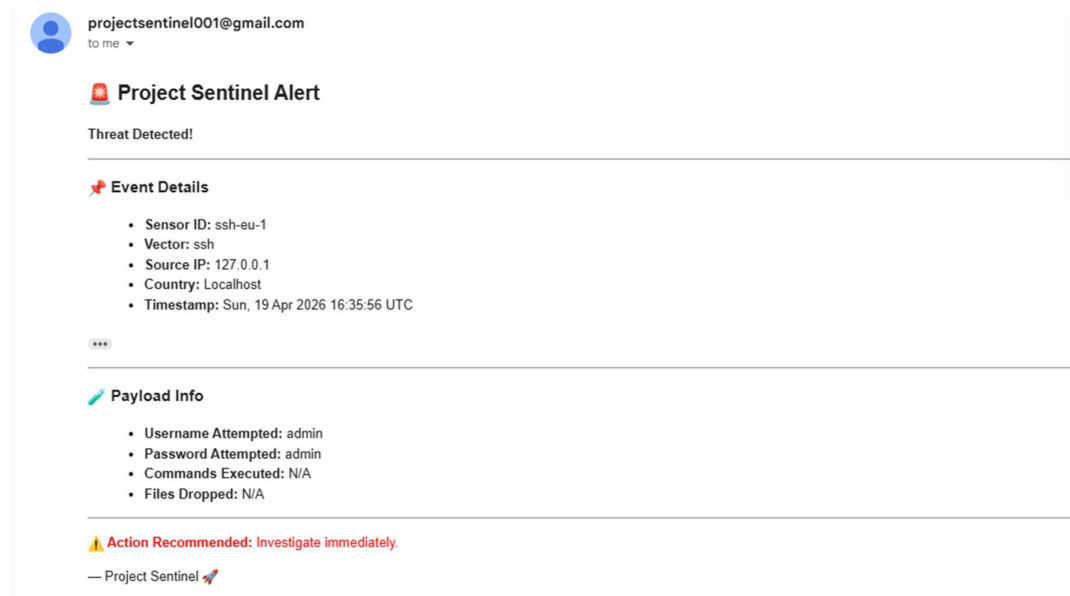


Figure 5.5(a)

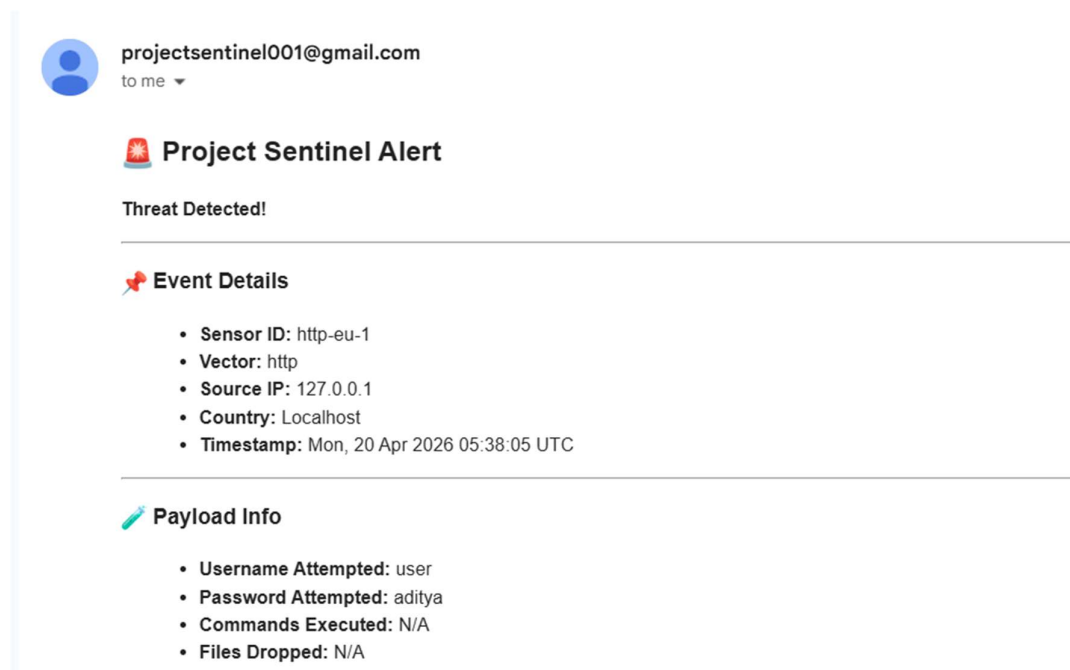


Figure 5.5(b)

5.6 IP Blocking Implementation

The IP blocking module is used to prevent repeated attacks from the same source.

Working Principle

When an IP address exceeds the allowed number of login attempts, it is automatically blocked using firewall rules.

Sample Code :

```
import os

def block_ip(ip):

    os.system(f'sudo iptables -A INPUT -s {ip} -j DROP')
```

Explanation

- Uses system-level firewall command
- Adds rule to block incoming traffic from specific IP
- Enhances system security

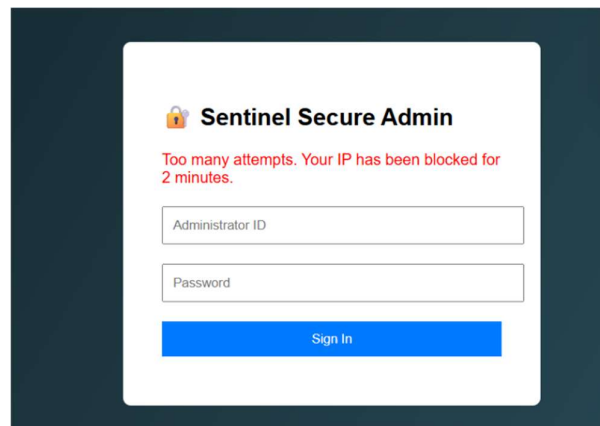


Figure 5.6

5.7 Grafana Dashboard Integration

The system integrates with Grafana to provide visualization of attack data.

Working Principle

The logged data is fed into the dashboard, where it is displayed in graphical form.

Features

- Real-time attack monitoring
- Graphical representation of data
- IP tracking
- Time-based analysis

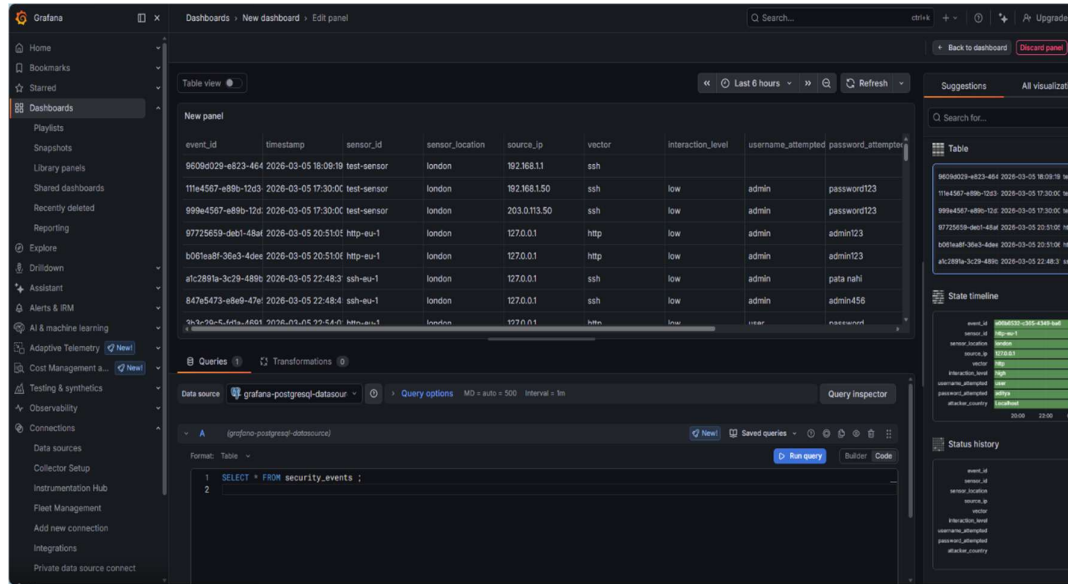


Figure 5.7

5.8 Key Features Implemented

The system successfully implements the following features:

- SSH Honeypot Simulation
- Real-time Logging
- Alert System with Email Notification
- Automated IP Blocking
- Data Visualization using Dashboard

5.9 Challenges Faced During Implementation

During development, several challenges were encountered:

- Handling real-time network connections

- Preventing system crashes due to multiple connections
- Managing log file size and structure
- Configuring firewall rules correctly
- Integrating visualization tools

5.10 Solutions to Challenges

To overcome the challenges:

- Efficient socket handling was implemented
- Logging mechanisms were optimized
- Firewall rules were tested carefully
- Modular design was adopted for easier debugging

CHAPTER 6: TESTING AND RESULTS

6.1 Introduction

Testing is an essential phase in the development of any system, as it ensures that all components function correctly and meet the desired objectives. In the "Project Sentinel - Honeypot Monitoring System", testing is performed to verify the effectiveness of the honeypot in detecting, logging, and responding to cyberattacks.

The system is tested under different scenarios, including unauthorized login attempts, repeated attack patterns, and web-based requests. The goal of testing is to ensure that the system can handle real-world attack situations efficiently.

6.2 Testing Methodology

The system is tested using a combination of manual and simulated attack methods. Different types of testing are performed to evaluate various aspects of the system.

Types of Testing Performed :

1. Functional Testing

This ensures that all modules of the system are working as expected.

- SSH honeypot accepts connections
- HTTP server responds to requests
- Logs are generated correctly
- Alerts are triggered properly

2. Security Testing

This verifies the system's ability to detect and respond to attacks.

- Detection of brute-force login attempts
- Identification of suspicious IP addresses
- Blocking of malicious users

3. Performance Testing

This evaluates how the system performs under multiple attack attempts.

- Handling multiple connections simultaneously
- Efficient logging without delay

- Stable system behavior

6.3 Test Cases

The following test cases were designed and executed to validate system functionality:

Table 6.1: Test Cases

Test Case ID	Description	Expected Result	Actual Result	Status
TC1	SSH connection attempt	Connection logged	Successful	Pass
TC2	Wrong SSH credentials	Credentials logged	Successful	Pass
TC3	Multiple login attempts	Alert generated	Successful	Pass
TC4	Repeated attacks	IP blocked	Successful	Pass
TC5	HTTP request	Request logged	Successful	Pass
TC6	Suspicious HTTP activity	Alert triggered	Successful	Pass

6.4 Testing of SSH Honeypot

The SSH honeypot was tested by attempting multiple login attempts using incorrect credentials.

Observations:

- The system successfully captured IP addresses
- Username and password attempts were logged
- Repeated attempts triggered alerts

```
MINGW64; c:/Users/chaya/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ cd sensors/ssh

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh (main)
$ python main.py
[+] Sentinel SSH Honeypot listening on port 2222...
[*] Incoming SSH connection from 127.0.0.1

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: sdbcndhg
[DEBUG SSH] 127.0.0.1 attempts: 1
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: jdhjdjhfdgdhdf
[DEBUG SSH] 127.0.0.1 attempts: 2
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: uefbgjskdssf
[DEBUG SSH] 127.0.0.1 attempts: 3
[*] Sending SSH event to API...
[DEBUG] 200 + {"status": "success"}
[+] Event sent successfully
[BLOCKED] SSH IP 127.0.0.1 blocked for 2 minutes
```

Figure 6.1

6.5 Testing of HTTP Honeypot

The HTTP server was tested by sending requests through a browser or tools such as curl.

Observations:

- HTTP requests were successfully captured
- Request details such as method and URL were logged
- Suspicious requests triggered alerts

```
MINGW64; c:/Users/chaya/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/http

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ cd sensors/http

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/http (main)
$ python main.py
[+] Sentinel HTTP Honeypot running on http://localhost:8080
* Serving Flask app 'main'
* Debug mode: off

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: hshsgdh
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 1
[*] Sending event to API...
[-] Connection error: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: bhzcz
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 2
[*] Sending event to API...
[-] Connection error: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):
```

Figure 6.2

6.6 Alert System Testing

The alert system was tested by generating repeated login attempts.

Observations:

- Alerts were triggered after threshold exceeded
- Notifications were displayed in real-time
- Email alerts (if implemented) were successfully sent

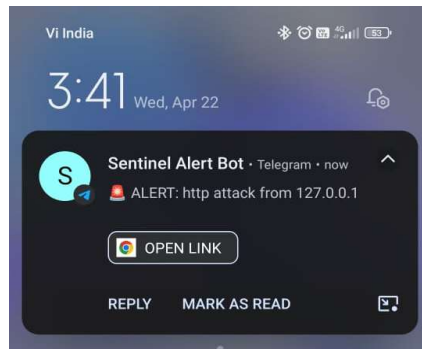


Figure 6.3(a)

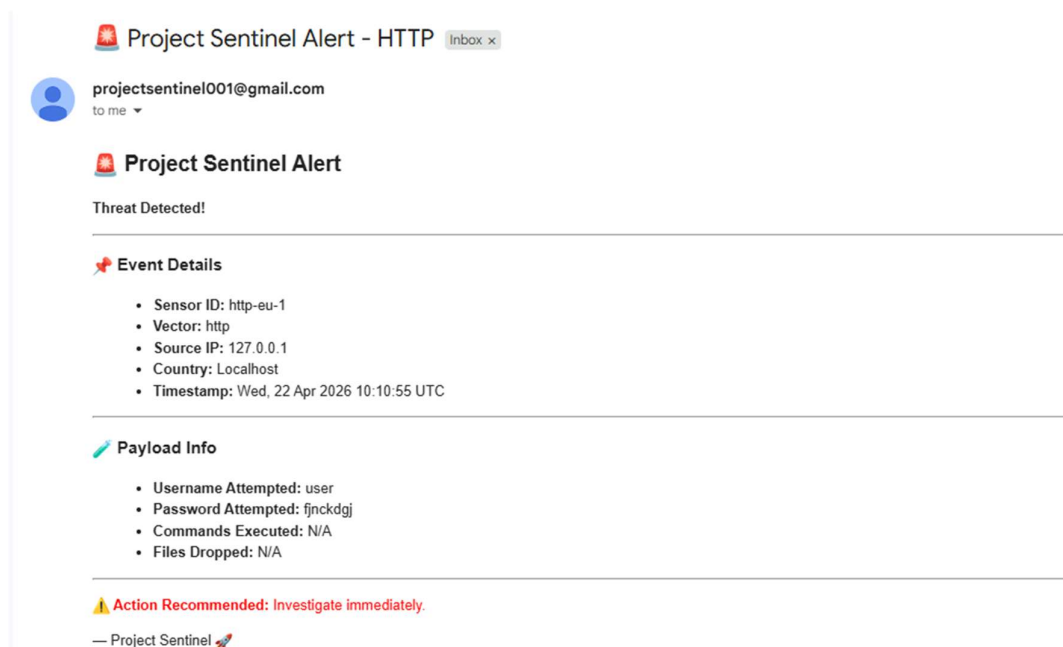


Figure 6.3(b)

6.7 IP Blocking Testing

The IP blocking module was tested by exceeding login attempt limits.

Observations:

- IP address was successfully blocked
- Further attempts from same IP were denied
- Firewall rules were applied correctly

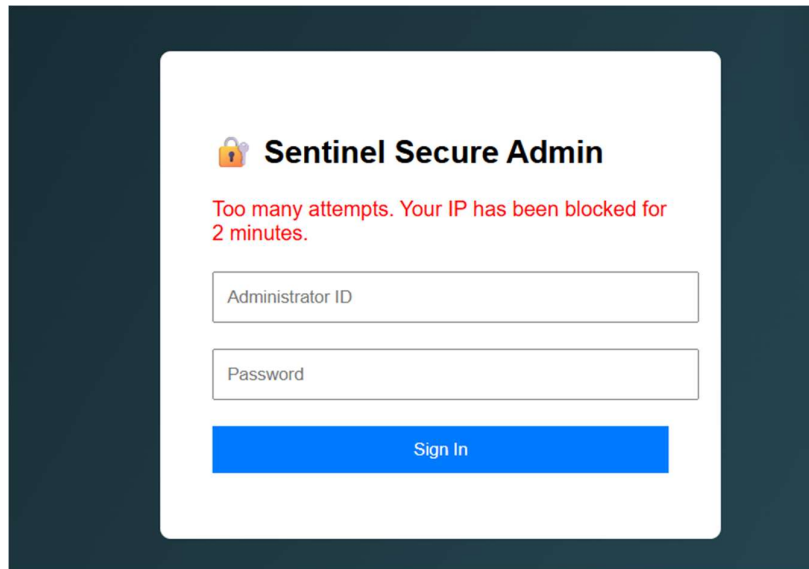


Figure 6.4

6.8 Graphical Results (Dashboard Analysis)

The Grafana dashboard was used to visualize attack data.

Displayed Metrics:

- Number of attacks over time
- Distribution of IP addresses
- Frequency of login attempts
- HTTP request trends

No. of attacks over time:



Figure 6.5

Frequency of login attempts :



Figure 6.6

HTTP request trends :

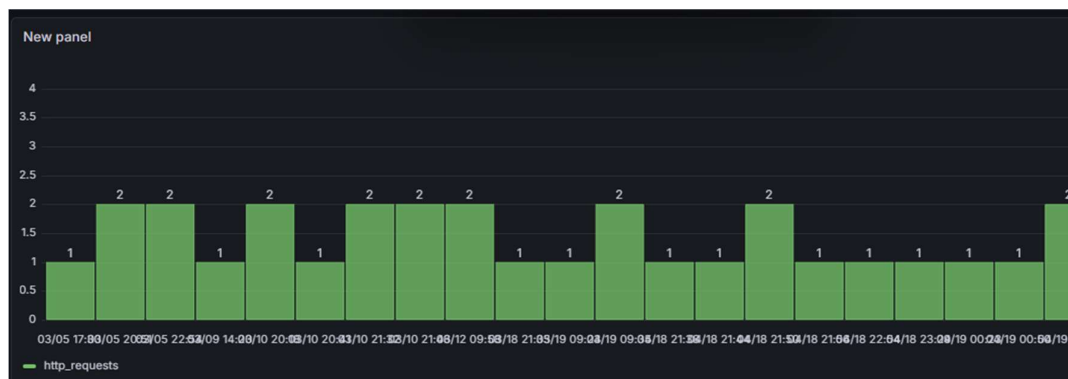


Figure 6.7

Analysis of Results

- Attack frequency increased during testing phase
- Certain IP addresses showed repeated attempts

- Alerts and blocking mechanisms reduced attack rate

6.9 Performance Evaluation

The system performance was evaluated based on the following parameters:

- **Response Time:** Fast response to incoming connections
- **Logging Efficiency:** No data loss observed
- **System Stability:** No crashes during testing

6.10 Limitations Identified During Testing

During testing, the following limitations were observed:

- Limited to simulated attack scenarios
- No advanced machine learning detection
- Dependency on system firewall configuration

6.11 Summary of Results

The system successfully achieved its objectives:

- Detected unauthorized access attempts
- Logged attacker activities
- Generated alerts in real time
- Blocked malicious IP addresses
- Visualized attack data effectively

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

The "Project Sentinel - Honeypot Monitoring System" successfully demonstrates the implementation of a multi-service honeypot system designed to monitor, detect, and analyze cyberattacks. The system effectively simulates vulnerable services such as SSH and HTTP to attract attackers and capture their activities in a controlled environment.

Unlike traditional security systems that focus primarily on prevention, this project adopts a proactive approach by allowing attackers to interact with a decoy system. This enables the collection of valuable information about attack patterns, techniques, and behaviors.

Throughout the implementation and testing phases, the system proved to be capable of:

- Capturing unauthorized access attempts through SSH and HTTP services
- Logging attacker credentials, IP addresses, and request details
- Generating real-time alerts for suspicious activities
- Sending notifications (including email alerts where implemented)
- Automatically blocking malicious IP addresses using firewall rules
- Visualizing attack data using a Grafana dashboard

The integration of multiple modules such as sensors, core API, logging system, alert mechanism, and visualization dashboard demonstrates a well-structured and modular architecture. This design ensures scalability and allows future enhancements without major changes to the system.

Overall, the project achieves its primary objective of creating an effective honeypot monitoring system and provides a strong foundation for further research and development in cybersecurity.

7.2 Achievements of the Project

The major achievements of the project include:

- Successful development of a multi-service honeypot system
- Implementation of SSH and HTTP attack detection
- Real-time logging of attacker activities
- Integration of alert system with notification capabilities
- Automated IP blocking using firewall rules
- Deployment-ready architecture using modular design
- Integration with Grafana for real-time visualization

These achievements demonstrate the practical applicability of the system in real-world cybersecurity scenarios.

7.3 Limitations

Despite its effectiveness, the system has certain limitations:

- The honeypot is limited to SSH and HTTP services only
- It cannot detect attacks that do not interact with the honeypot
- Basic rule-based detection is used instead of advanced AI techniques
- Email alert system may require additional configuration
- System performance may vary under heavy attack loads

These limitations highlight areas where further improvements can be made.

7.4 Future Scope

The project can be enhanced in several ways to improve its functionality and effectiveness:

- Integration of Machine Learning for intelligent attack detection
- Support for additional protocols such as FTP, SMTP, and DNS
- Deployment on cloud platforms for scalability

- Implementation of advanced alert systems (SMS, push notifications)
- Development of a web-based control panel for system management
- Integration with threat intelligence platforms
- Real-time attack prediction and analysis

These enhancements would make the system more robust and suitable for enterprise-level deployment.

REFERENCES:

1. Niels Provos and Thorsten Holz, "Virtual Honeypots: From Botnet Tracking to Intrusion Detection", Addison-Wesley.
2. The HoneyNet Project, "Know Your Enemy: Learning about Security Threats".
3. Cowrie Honeypot Documentation, <https://github.com/cowrie/cowrie>
4. Grafana Documentation, <https://grafana.com/docs/>
5. Python Official Documentation, <https://docs.python.org/3/>
6. Linux iptables Documentation, <https://linux.die.net/man/8/iptables>
7. OWASP Foundation, <https://owasp.org/>
8. Research papers on Honeypot Systems and Network Security (IEEE, Springer)

APPENDIX

APPENDIX A: SYSTEM LOG DATA

Description

This appendix contains sample logs generated by the honeypot system. These logs represent attacker activity captured by SSH and HTTP honeypot services. The logs include IP addresses, timestamps, attempted credentials, and request details.

Sample HTTP Attack Logs :

```
[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: fjnckdgj
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 1
[*] Sending event to API...
[-] Connection error: HTTPSPool(host='sentinel-ap

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: dacsffs
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 2
[*] Sending event to API...
[DEBUG] 200 → {"status":"success"}
[+] Event sent successfully

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: hcjdhdkj
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 3
[*] Sending event to API...
[DEBUG] 200 → {"status":"success"}
[+] Event sent successfully

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: udgdjvgv
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 4
[*] Sending event to API...
[DEBUG] 200 → {"status":"success"}
[+] Event sent successfully

[!] HTTP Login Attempt from 127.0.0.1
User: user | Pass: rucvfk
Country: Localhost
[DEBUG] 127.0.0.1 attempts: 5
[*] Sending event to API...
[DEBUG] 200 → {"status":"success"}
[+] Event sent successfully
[BLOCKED] HTTP IP 127.0.0.1 blocked for 2 minutes
```

Fig. 8.1

Sample SSH Attack Logs :

```
MINGW64/c/Users/chaya/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ cd sensors/ssh

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh (main)
$ python main.py
[!] Sentinel SSH Honeypot listening on port 2222...
[*] Incoming SSH connection from 127.0.0.1

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: sdbcmdhg
[DEBUG SSH] 127.0.0.1 attempts: 1
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: jdhdjhfdgdhdf
[DEBUG SSH] 127.0.0.1 attempts: 2
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: uefbgjskdssf
[DEBUG SSH] 127.0.0.1 attempts: 3
[*] Sending SSH event to API...
[DEBUG] 200 + {"status": "success"}
[+] Event sent successfully
[BLOCKED] SSH IP 127.0.0.1 blocked for 2 minutes
```

Fig. 8.2

Analysis

- Multiple login attempts indicate brute-force attack
- Requests to /admin and /login indicate vulnerability scanning
- Logs help identify suspicious IP patterns

APPENDIX B: DATABASE STRUCTURE

Description

The system uses a database to store logs, alerts, and blocked IP addresses. The schema ensures structured storage and efficient retrieval.

Table B.1: attack_logs

Column Name	Data Type	Description
id	INT	Unique ID
ip_address	VARCHAR	Attacker IP
service	VARCHAR	SSH / HTTP
data	TEXT	Attack details
timestamp	DATETIME	Time of attack

Table B.2 : blocked_ips

Column Name	Data Type	Description
Id	INT	Unique ID
Ip_address	VARCHAR	Blocked IP
reason	TEXT	Reason for blocking
Blocked_at	DATETIME	Time of blocking

APPENDIX C: API ENDPOINTS

Description

The core API handles logging, alerting, and monitoring functionalities.

Table C.1 : API Endpoints

Method	Endpoint	Description
POST	/api/log	Store attack log
GET	/api/logs	Retrieve logs
POST	/api/block	Block IP
GET	/api/alerts	Get alerts

Example API Request :

POST /api/log

```
{
  "ip": "192.168.1.10",
  "service": "SSH",
  "data": "admin / 123456"
```

}

APPENDIX D: PROJECT FOLDER STRUCTURE

Description

The project follows a modular architecture separating API, sensors, and infrastructure.

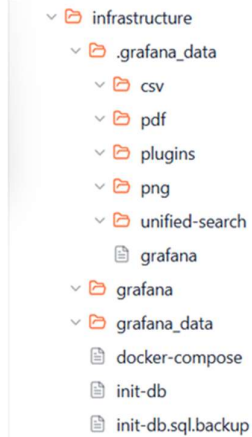


Fig. 8.3(a)

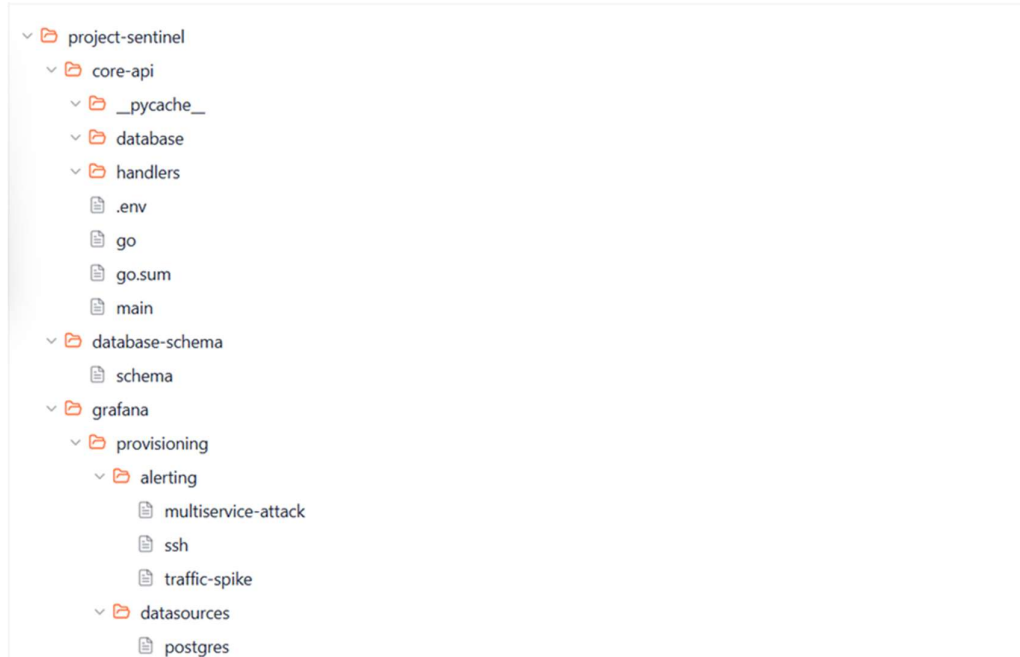


Fig. 8.3(b)



Fig. 8.3(c)

Explanation

- **core-api** → Handles backend logic
- **sensors** → Captures attacker activity
- **grafana** → Visualization and monitoring
- **infrastructure** → Deployment setup

APPENDIX E: COMMANDS USED

Description :

This appendix provides a list of commands used during the development, deployment, and testing of the "Project Sentinel - Honeypot Monitoring System". These commands were used to run different components of the system, simulate attacks, and manage SSH connections.

Table E.1 :Command Execution

Command	Purpose
<code>python main.py</code>	Starts the Python-based honeypot sensor (SSH/HTTP)
<code>go run main.go</code>	Runs the core API backend service written in Go
<code>docker compose up</code>	Starts all services (Grafana, database, etc.) using Docker
<code>ssh admin@localhost -p 2222</code>	Simulates an SSH attack attempt on the honeypot

```
ssh-keygen -R "[localhost]:2222"
```

Removes old SSH host key entry to avoid connection errors

1. Running Honeypot Sensors

```
python main.py
```

This command is used to start the honeypot sensor module written in Python. It activates the SSH and HTTP services which listen for incoming attacker connections.

Function:

- Captures login attempts
- Logs attacker credentials
- Sends data to the backend

2. Running Core API (Backend Service)

```
go run main.go
```

This command runs the backend server implemented in Go. The core API handles:

- Data processing
- Logging management
- Communication with database
- Alert generation

3. Starting Infrastructure using Docker

```
docker-compose up
```

This command initializes all required services using Docker containers.

Services Started:

- Grafana dashboard
- Database (PostgreSQL)
- Supporting infrastructure

This ensures that the system runs in a consistent and isolated environment.

4. Simulating SSH Attack

```
ssh admin@localhost -p 2222
```

This command is used to simulate an attacker attempting to log into the honeypot.

Explanation:

- admin → fake username
- localhost → target system
- -p 2222 → custom SSH port used by honeypot

This generates login attempts that are captured and logged by the system.

5. Fixing SSH Connection Error

```
ssh-keygen -R "[localhost]:2222"
```

This command removes the stored SSH key for the specified host and port.

Purpose:

- Prevents "WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED" error
- Allows reconnecting to the honeypot without conflicts

APPENDIX F: ALERT SYSTEM WORKFLOW

Description

The alert system detects suspicious behavior and triggers notifications.

Workflow Steps

1. Attack detected by sensor
2. Data sent to API
3. API checks threshold
4. Alert generated
5. Email notification sent to user
6. IP blocked if necessary

Output Example :

```
MINGW64/c/Users/chaya/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel (main)
$ cd sensors/ssh

chaya@Victus MINGW64 ~/OneDrive/Desktop/Project_Sentinel/project-sentinel/sensors/ssh (main)
$ python main.py
[!] Sentinel SSH Honeypot listening on port 2222...
[*] Incoming SSH connection from 127.0.0.1

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: sdbcmdhg
[DEBUG SSH] 127.0.0.1 attempts: 1
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: jdhdjhfdgdhdf
[DEBUG SSH] 127.0.0.1 attempts: 2
[*] Sending SSH event to API...
[-] Failed to connect to Core API: HTTPSConnectionPool(host='sentinel-api-6ojq.onrender.com', port=443):

[!] SSH Login Attempt from 127.0.0.1
Country: Localhost
User: admin | Pass: uefbgjskdssf
[DEBUG SSH] 127.0.0.1 attempts: 3
[*] Sending SSH event to API...
[DEBUG] 200 → {"status": "success"}
[+] Event sent successfully
[BLOCKED] SSH IP 127.0.0.1 blocked for 2 minutes
```

Fig. 8.4